

Ansys Mechanical – Générateur de rapport PowerPoint

Script IronPython 2.7 exécuté directement dans la **console de scripting d'Ansys Mechanical**. Il ouvre une fenêtre WPF dans laquelle l'ingénieur sélectionne les éléments du modèle (géométrie, maillage, conditions aux limites, contacts, résultats...) à inclure dans le rapport, puis génère automatiquement une présentation PowerPoint à partir d'un template corporate, avec archivage des données extraites en CSV.

Propriété et confidentialité. Ce dépôt est la propriété exclusive de **Liebherr Components Colmar SAS**. Tous droits réservés. Toute redistribution, copie ou réutilisation de ce code, en tout ou partie, sans l'accord écrit préalable de Liebherr Components Colmar SAS, est strictement interdite. Voir [LICENSE](#).

Documentation assistée par IA

Une IA (DeepWiki) est disponible pour explorer et comprendre ce code :

- **Discussion** : <https://deepwiki.com/YannLiebherrInternship/Ansys-Report-Automation>
- **Dépôt GitHub source** : <https://github.com/YannLiebherrInternship/Ansys-Report-Automation>

Sommaire

1. [Prérequis](#)
2. [Installation](#)
3. [Structure du dépôt](#)
4. [Pipeline de données](#)
5. [Modules `00\_constants.py` → `05\_interactive\_slides.py`](#)
6. [Interface WPF \(`AnsysReportGenerator\_WPF.py` / `.xaml`\)](#)
7. [XAML : mise en page déclarative et lien avec Python](#)
8. [Notions métier Ansys utilisées dans le code](#)
9. [API Ansys Mechanical utilisées](#)
10. [Comment le code pilote PowerPoint \(COM Interop\)](#)
11. [Raisonnements Python employés dans le projet](#)
12. [Bases Python illustrées par le code du projet](#)
13. [Créer une nouvelle slide personnalisée dans le Master Template](#)
14. [Pièges connus / choix techniques](#)

1. Prérequis

Élément	Détail
Ansys Mechanical	2023 ou supérieur (fournit IronPython 2.7 embarqué + l'API <code>ExtAPI/DataModel</code>)
Microsoft Office	PowerPoint installé (COM Interop <code>Microsoft.Office.Interop.PowerPoint</code>)
Template PowerPoint	Fichier <code>.pptx</code> corporate avec les layouts personnalisés attendus (voir \$5, <code>00_constants.py</code>)
Système	Windows (Windows Forms + WPF via .NET, COM Interop)

Aucune dépendance Python externe n'est nécessaire : tout passe par la bibliothèque standard IronPython (`os`, `csv`, `re`, `datetime`, `shutil`, `xml.etree.ElementTree`) et par des assemblies .NET chargées via `clr.AddReference()`.

2. Installation

Deux méthodes sont disponibles. Dans les deux cas, l'interface obtenue (voir §6) est strictement identique — seule la méthode de déploiement et de lancement change.

	Méthode 1 — Bouton manuel	Méthode 2 — Extension <code>.wbex</code>
Installation	Copie manuelle d'un dossier, projet par projet	Une seule installation (Extension Manager), valable pour tous les projets
Chemins de fichiers	Calculés au premier lancement à partir d'un dossier <code>Report Generator</code> créé à la main	Gérés automatiquement par l'extension, qui détecte seule le projet Workbench hôte
Accès	Bouton "promu" dans l'onglet Automation, persistant entre les sessions d'un même projet	Onglet dédié, intégré à l'extension
Nettoyage des fichiers	Onglet "Files" de l'application (voir §6)	Onglet "Files" de l'extension — manuel, vérification finale requise
Désinstallation	Suppression manuelle du dossier <code>Report Generator</code>	Automatique : l'extension supprime tous les fichiers qui lui sont associés

2.1 Méthode 1 — Bouton manuel dans la console de scripting

1. Ouvrir le projet Ansys concerné et l'enregistrer au moins une fois. Cela crée un dossier `<NomDuProjet>_files`, qui contient un sous-dossier `user_files`.
2. À côté de `user_files` (et non dedans), créer un dossier nommé exactement `Report Generator`.
3. Copier à plat dans ce dossier, sans sous-dossier :
 - `00_constants.py`, `01_data_export.py`, `02_image_export.py`, `03_ppt_utils.py`, `04_slides.py`, `05_interactive_slides.py`
 - `AnsysReportGenerator_WPF.py`, `AnsysReportGenerator_WPF.xaml`
 - `Master Template_def.pptx`
 - `README.md`, `README_EN.md`
 - le dossier `logo/` (logo affiché dans l'interface)

Les sous-dossiers de données (`data/image_export`, `data/csv_export`, `data/reports`, `data/export_3D`) n'existent pas encore à ce stade : ils sont créés automatiquement au premier lancement.

4. Ouvrir Workbench, puis depuis Workbench ouvrir Mechanical, puis dans Mechanical ouvrir la console de scripting (onglet **Automation**).

- 5. Dans la console, parcourir les fichiers et ouvrir `AnsysReportGenerator_WPF.py` depuis le dossier `Report Generator` qui vient d'être créé.
- 6. Faire un premier test en cliquant sur **"Run script"**, pour vérifier que la fenêtre s'ouvre correctement.
- 7. Si tout fonctionne, promouvoir le script en bouton : dernier bouton de la console, **"Show Button Editor"**, puis **"Promote script to button"**. Le bouton apparaît alors dans la barre de navigation de Mechanical, sous **Automation**, et reste accessible entre les sessions d'un même projet.

L'interface graphique — et donc ce bouton — est actuellement uniquement en anglais.

Aucun chemin n'est à modifier dans le code pour que cela fonctionne sur un nouveau projet ou un nouveau poste : tous les chemins de travail sont recalculés automatiquement à partir de l'emplacement de `Report Generator` (`ExtAPI.DataModel.Project.ProjectDirectory`). Seul le template PowerPoint doit être fourni manuellement, puisque c'est un fichier de contenu qui ne peut pas être généré automatiquement ; s'il est absent au chargement, un avertissement s'affiche dans la console, et la génération échouera proprement (message clair, pas de plantage) tant qu'un template valide n'aura pas été déposé au bon endroit. Ces chemins restent modifiables sans relancer le script, depuis l'onglet "Files" de l'interface (bouton "Reset" pour revenir aux valeurs calculées automatiquement).

2.2 Méthode 2 — Extension Ansys (`.wbex`)

- Installation via l'**Extension Manager** d'Ansys, à partir du fichier `.wbex`.
- Une fois installée, l'extension gère automatiquement ses propres chemins de fichiers : elle récupère elle-même la destination du projet Ansys Workbench dans lequel elle est installée — aucun dossier à créer, aucun chemin à configurer manuellement.
- Un onglet **"Files"** de l'extension permet de gérer ces chemins de fichiers et d'accéder aux résultats générés.
- **Nettoyage** : possible depuis ce même onglet, mais **manuel** — il nécessite une vérification finale de l'utilisateur avant suppression ; l'extension ne déclenche jamais ce nettoyage d'elle-même.
- **Désinstallation** : à la désinstallation, l'extension supprime automatiquement tous les fichiers qui lui sont associés. Penser à sauvegarder tout export à conserver avant de désinstaller.

3. Structure du dépôt

Le dossier de ce dépôt correspond au contenu du dossier `Report Generator` à déployer pour la Méthode 1 (§2.1).

Fichier / dossier	Rôle
<code>AnsysReportGenerator_WPF.py</code>	Point d'entrée — seul fichier exécuté directement dans Mechanical
<code>AnsysReportGenerator_WPF.xaml</code>	Mise en page de la fenêtre principale (voir §7)
<code>00_constants.py</code> → <code>05_interactive_slides.py</code>	Les six modules, chargés dans l'ordre par <code>execfile()</code> (voir §5)
<code>Master Template_def.pptx</code>	Template corporate — seul chemin non créé automatiquement

Fichier / dossier	Rôle
logo/	Logo affiché dans l'interface
README.md / README_EN.md	Cette documentation

Un dossier `data/` est créé automatiquement au premier lancement (absent au départ sur un nouveau projet) et contient quatre sous-dossiers :

Sous-dossier	Contenu
image_export/	Images PNG exportées (viewport, graphiques reconstruits)
csv_export/	CSV archivés indépendamment du PowerPoint
reports/	Copies de travail du template + rapports .pptx générés
export_3D/	Fichiers .avz (vues 3D interactives), générés par le bouton "Export to 3D"

Le dossier des légendes (`legend/`) n'est **pas** dans `data/` : il vit à côté, dans `user_files/legend` du projet Ansys — voir §6.

`04_slides.py` et `05_interactive_slides.py` coexistent volontairement : `04_slides.py` fournit les fonctions `create..._slide` d'origine, qui exportent toujours tout sans configuration possible, et `05_interactive_slides.py` les réutilise comme briques de base pour construire des versions filtrées par la sélection de l'utilisateur (`build..._slides`), sans dupliquer la logique d'extraction CSV/image déjà écrite. L'application WPF n'appelle que les fonctions de `05_interactive_slides.py`, à l'exception de `create_geometry_slide` et `create_analysis_parameters_slide` de `04_slides.py`, réutilisées telles quelles.

4. Pipeline de données

Génération d'un rapport, quelle que soit la catégorie de slide concernée :

- Extraction** — depuis l'arbre Mechanical / le Tabular Data pane vers CSV (`01_data_export.py`, fonctions `export*_csv`, écrites dans `CSV_EXPORT_FOLDER`), ou depuis le viewport 3D vers PNG (`02_image_export.py`, `export_current_view_image/export_object_image/export_chart_image_from_csv`, écrits dans `IMAGE_EXPORT_FOLDER`).
- Construction** — `PPTReportBuilder` (`03_ppt_utils.py`) copie d'abord le template dans `REPORT_OUTPUT_FOLDER` (jamais l'original), l'ouvre via COM Interop, puis chaque appel à une méthode `add..._slide` ajoute une slide en y insérant l'image et/ou la table lues aux étapes précédentes.
- Sauvegarde** — une fois toutes les slides ajoutées, la présentation est enregistrée sous son nom final dans `REPORT_OUTPUT_FOLDER` : c'est le rapport livré à l'utilisateur.

Le CSV est toujours conservé sur disque, indépendamment de son insertion réussie ou non dans le PowerPoint : il reste consultable et téléchargeable depuis l'onglet "Files" de l'interface, et constitue une archive exploitable séparément du rapport. Son insertion en table PowerPoint est simplement ignorée si le tableau dépasse `MAX_TABLE_ROWS/MAX_TABLE_COLUMNS` (`00_constants.py`), un tableau aussi grand devenant illisible une fois inséré dans une slide.

5. Modules `00_constants.py` → `05_interactive_slides.py`

`00_constants.py`

Chemins racine, index des layouts personnalisés du template (`LAYOUT_IMAGE_TABLE`, `LAYOUT_TABLE_ONLY`, `LAYOUT_MESH_MULTI`), limites d'affichage des tableaux, et helpers génériques indépendants d'Ansys : `ensure_folder_exists`, `safe_file_name`, `get_unique_file_path`, `clean_cell_text`, `to_csv_cell`. Doit être exécuté en premier — toutes les constantes qu'il définit (`IMAGE_EXPORT_FOLDER`, `CSV_EXPORT_FOLDER`, etc.) sont utilisées telles quelles (variables globales, pas d'`import`) par tous les autres modules.

`01_data_export.py`

Tout ce qui lit le **Tabular Data pane** ou le modèle et écrit un CSV : données tabulaires d'un objet actif (`export_active_tabular_data`), résumé des contacts, rapport de maillage, matériaux utilisés (via le module Ansys `materials`), tableaux des paramètres de steps et des infos de résolution (`export_analysis_settings_csv/export_solution_info_csv`, utilisés dans la slide "Analysis Parameters").

`02_image_export.py`

Capture d'image du viewport Mechanical (`export_current_view_image`, basé sur `ExtAPI.Graphics.ExportImage`), et export "haut niveau" par type d'objet (géométrie, maillage, vue d'ensemble d'analyse, objet quelconque via un snapshot `Figure`). Contient aussi un moteur de tracé de graphique 2D minimal en `System.Drawing` (`export_chart_image_from_csv`) : les trackers de "Solution Information" n'ont pas de représentation 3D, leur graphique est donc redessiné à partir du CSV exporté plutôt que capturé depuis le viewport.

`03_ppt_utils.py`

Classe **PPTReportBuilder** : encapsule l'unique session COM PowerPoint ouverte sur la copie de travail du template, et expose les méthodes de haut niveau pour ajouter une slide (`add_image_table_slide`, `add_table_slide`, `add_analysis_context_slide`, `add_csv_table`, `save_as`, `close`). Voir §10 pour le détail de son fonctionnement interne.

`04_slides.py`

Fonctions `create_..._slide(report)` "historiques" : chacune traite **tous** les objets d'une catégorie du modèle (aucune sélection/configuration possible). Utilisées par l'UI pour Géométrie et Contexte d'analyse (cases indépendantes, pas de liste à cocher).

`05_interactive_slides.py`

Le plus gros module (~1800 lignes). Fournit toute la logique de support de l'interface :

Domaine	Contenu
Nettoyage	<code>remove_stale_figures()</code> — supprime les objets <code>Figure</code> résiduels d'une génération précédente

Domaine	Contenu
Export 3D (.avz)	<code>export_all_3d_views()</code> — pour chaque analyse, exporte chaque résultat simple et chaque enfant de Contact Tool / Bolt Tool de la branche <i>Solution</i> dans <code>EXPORT_3D_FOLDER</code>
Vue / coupe / échelle / légende par ligne	<code>apply_view_if_exists</code> , <code>apply_section_plane</code> , <code>apply_scale_factor</code> , <code>apply_legend_if_exists</code> — appliqués juste avant la capture d'un objet, puis réinitialisés juste après
Steps et slides combinées	<code>evaluate_result_for_step</code> (step par step) ; <code>add_multi_step_image_slide</code> (une slide combinée si un template existe pour le nombre exact de steps — <code>MULTI_STEP_SLIDE_TEMPLATES</code> : 2, 3, 4, 6 ou 8, sinon repli automatique en mode individuel)
Classes *RowConfig	<code>SlideRowConfig</code> , <code>GeometryPartRowConfig</code> , <code>MeshPartRowConfig</code> , <code>ContactRowConfig</code> , <code>SolutionInfoRowConfig</code> , <code>AnalysisContextRowConfig</code> — une instance par ligne de sélection dans l'UI
Collecteurs	<code>collect_views</code> , <code>collect_section_planes</code> , <code>collect_bodies</code> , <code>collect_boundary_conditions[_multi]</code> , <code>collect_bolt_pretensions[_multi]</code> , <code>collect_contact_tool_results[_multi]</code> , <code>collect_bolt_tool_results[_multi]</code> , <code>collect_all_results[_multi]</code> , <code>collect_solution_information_trackers[_multi]</code> , <code>collect_analyses...</code> — les variantes <code>_multi</code> compilent les objets de toutes les analyses du projet sous forme de tuples (objet, analyse)
Constructeurs "sélection-aware"	<code>build_bc_slides</code> , <code>build_bp_slides</code> , <code>build_result_slides</code> , <code>build_geometry_part_slides</code> , <code>build_mesh_part_slides</code> , <code>build_contact_summary_slide</code> , <code>build_solution_info_slides</code> , <code>build_analysis_context_slides</code> , <code>build_mesh_slide</code> — équivalents de <code>04_slides.py</code> mais limités à la sélection cochée
Géométrie par pièce isolée	<code>isolate_body_by_transparency</code> — pièce opaque, autres semi-transparentes, une slide par pièce
Mesh par pièce isolée	<code>show_only_body</code> — masque totalement les autres pièces ; jusqu'à 4 pièces par slide (<code>LAYOUT_MESH_MULTI</code>), au-delà une nouvelle slide démarre automatiquement

6. Interface WPF

`AnsysReportGenerator_WPF.py` définit la classe **ReportGeneratorApp**, qui charge `AnsysReportGenerator_WPF.xaml` via `XamlReader` et pilote une barre d'outils utilitaire (au-dessus des onglets) et 6 onglets verticaux, sur le côté gauche de la fenêtre.

Barre d'outils utilitaire — 4 actions globales, indépendantes de la sélection en cours :

Bouton	Action
Delete figures	Nettoie les objets Figure résiduels d'une génération précédente (<code>remove_stale_figures</code>)
Reset legends	Remet la légende du viewport à l'automatique (<code>reset_legend</code>)
Create basic views	Crée 7 vues (X+/X-/Y+/Y-/Z+/Z-/ISO) dans le View Manager, réutilisables dans le panneau latéral "..." (<code>create_basic_views</code>)
Export to 3D (.avz)	Exporte, pour chaque analyse, une vue 3D interactive .avz de chaque résultat simple et de chaque enfant de Contact Tool / Bolt Tool de la branche <i>Solution</i> (<code>export_all_3d_views</code> , voir §9), dans <code>data/export_3D/</code>
Onglet	Contenu
General slides	"Overview slides" : deux cartes distinctes Geometry / Mesh (case à cocher + statut + bouton "Settings"), "Parts to isolate (geometry)", "Mesh part to isolate", "Analysis context" (une ligne par analyse du projet, avec sélection de vue)
Conditions and contacts	Boundary Conditions, Bolt Pretension, Contacts to display, Connection: Contact Tool (branche <i>Connections</i> , sans step), Solution Information
Result categories	Contact Tool Results (branche <i>Solution</i> , avec steps), Results, Bolt Tool
Combined slide	Construction d'une slide combinée "différents résultats" — voir ci-dessous
Report preview	Une carte par catégorie cochée (ou slide combinée ajoutée), réorganisable par glisser-déposer — l'ordre choisi est l'ordre de génération du rapport
Files	Chemins modifiables (template, images, CSV, légendes, rapports), nettoyage des dossiers de données (voir ci-dessous), liste des CSV déjà générés (Open/Show in folder), progression + accès au dernier rapport généré

Onglet "Combined slide (different results)". Ce flux vivait auparavant dans 3 boîtes de dialogue modales successives (choix du template, puis grille, puis choix de résultat) ; il est désormais entièrement intégré dans cet onglet, sans aucune fenêtre séparée.

- En haut : choix d'un template multi-image (2/3/4/6/8 résultats, mêmes `MULTI_STEP_SLIDE_TEMPLATES` que les slides combinées multi-step) et un bouton "Add to report".
- À gauche : une grille 2×4 où seules les N premières cases du template choisi sont actives ; cliquer sur une case vide affiche à droite la liste (filtrable) des résultats disponibles.
- À droite : cliquer sur un résultat bascule le panneau sur sa configuration graphique complète (mêmes champs qu'une ligne normale — vue/coupe/légende/apparence/scoping/scale factor — mais sans notion de step, un résultat différent et figé par case) ; le bouton "Apply" valide la case.

"Add to report" exige que toutes les cases actives soient configurées, puis ajoute la configuration (`MultiResultSlideConfig`) à `self._multi_result_slides` et réinitialise la grille pour en construire une autre — rien n'est généré immédiatement : comme les autres catégories, la slide apparaît comme une carte dans l'onglet "Preview" (bouton "Delete" dédié, pas de case à cocher) et n'est construite qu'au clic sur

"Generate report" (`ReportGeneratorApp._build_multi_result_tab`, méthodes `_on_multi_result_*/_show_multi_result_*`, `build_multi_result_slide/capture_multi_result_cell_image` dans `05_interactive_slides.py`).

Panneau latéral global de configuration ("..."). Chaque ligne de sélection possède un bouton "..." qui n'ouvre plus de fenêtre séparée : il affiche à droite de la fenêtre principale un panneau "SETTINGS" partagé par tous les onglets. Le contenu dépend du "kind" de la ligne cliquée (`ReportGeneratorApp._open_config_panel`) :

Kind	Champs
"result"	Vue / coupe / légende (fichier + orientation) / mode d'affichage des couleurs (Contour View) / affichage du scoping / échelle de déformation (manuel ou Auto Scale x1/x2) / sélection de steps
"geometry_part"	Vue / coupe / opacité du contexte (pièce isolée en géométrie)
"mesh_part"	Vue seulement (pièce isolée en mesh, mais aussi Géométrie/Maillage/Contexte d'analyse)
"solution_info"	Titre / axes / couleur du graphique reconstruit

Chaque catégorie de champs est un couple de fonctions partagées `_build_*_fields/_apply_*_fields`, qui posent/lisent leurs contrôles sur un `target` générique (`_ConfigFieldsHolder`, un simple sac d'attributs) plutôt que sur `self` d'une classe de fenêtre dédiée — ce découplage permet au même code de servir à la fois au panneau latéral global et au panneau de case de l'onglet "Combined slide". "Apply" valide (`row_config.configured = True`) et ferme le panneau ; "Cancel"/le bouton "x" ferment sans valider.

Ce même panneau peut aussi s'ouvrir en **mode groupe** : chaque en-tête de section propose un bouton "Configure selection..." qui applique en une fois les réglages choisis à toutes les lignes cochées de la section (`ReportGeneratorApp._on_bulk_config_click/_open_config_panel(..., bulk_rows=...)`) — une simple boucle Python sur les `row_config`, sans reconstruction de panneau ni appel API par ligne.

Sélection de vue pour Géométrie / Maillage / Contexte d'analyse. Ces trois cases disposent d'une vue (View Manager) sélectionnable : pour Géométrie et Maillage, un bouton "..." ouvre le panneau latéral en "mesh_part" sur un `MeshPartRowConfig` dédié (`self._geometry_view_config/self._mesh_view_config`) ; pour le Contexte d'analyse, `AnalysisContextRowConfig` porte un `view_name` avec son propre bouton "...". Dans les trois cas, la vue choisie est appliquée juste avant la capture (`apply_view_if_exists`), sans réinitialisation après.

Apparence des résultats (Contour View / légende / scoping / échelle de déformation). Le panneau en "result" expose quatre réglages par ligne :

- **Contour View** (`ResultPreference.ContourView`) : `ContourBands`, `Isolines`, `SmoothContours`, `SolidFill` — noms .NET conservés tels quels dans l'UI.
- **Orientation de légende** (`GlobalLegendSettings.LegendOrientation`) : Vertical ou Horizontal.
- **Scoping display** (`ResultPreference.ScopingDisplay`) : `ScopedBodies` (défaut), `ResultOnly`, `AllBodies`.
- **Échelle de déformation** : manuelle (facteur numérique) ou l'un des deux presets natifs "Auto Scale x1"/"Auto Scale x2" (`ResultPreference.DeformationScaling/DeformationScaleMultiplier`).

Par défaut, un résultat non configuré est capturé en `ContourBands`, légende verticale, scoping `ScopedBodies`, échelle manuelle x1. Ces réglages sont portés par `SlideRowConfig` et appliqués uniquement pendant la capture de la ligne concernée, puis systématiquement réinitialisés juste après (`reset_contour_view/reset_legend_orientation/reset_scoping_display/reset_scale_factor`), afin qu'un réglage choisi pour une ligne ne "fuie" jamais sur la suivante. Les fonctions `apply_*` appellent `ExtAPI.Graphics.Redraw()` juste après avoir modifié leur propriété : sans cet appel explicite, l'image exportée juste après continuerait de refléter l'ancien état.

Deux réglages s'appliquent en revanche globalement, à tous les exports d'image sans exception : `ModelColoring = ModelColoring.ByMaterial` (`set_material_display`, avant chaque export de géométrie/maillage), et `ShowLogo = False` (forcé dans `export_current_view_image`, pour qu'aucune image du rapport n'affiche le logo Ansys).

Cadrage de la caméra : responsabilité de l'utilisateur. Les fonctions d'export d'image n'appellent plus `ExtAPI.Graphics.Camera.SetFit()` avant la capture : cela écrasait silencieusement toute vue choisie par l'utilisateur. C'est donc à l'utilisateur de cadrer la vue (manuellement ou via une vue nommée) avant de générer le rapport. Seul `create_basic_views()` (bouton "Create basic views") continue d'utiliser `SetFit()`, puisqu'il sert à définir le cadrage des 7 vues standard.

La génération (`ReportGeneratorApp._on_generate`) parcourt l'ordre de l'onglet "Report preview", ouvre une unique session `PPTReportBuilder`, appelle la fonction `build..._slides` correspondant à chaque catégorie, met à jour la barre de progression (`SWF.Application.DoEvents()` pour garder la fenêtre réactive), puis ferme proprement la session PowerPoint et active les boutons "Open"/"Show in folder".

Onglet Files : disposition en 4 quadrants. Haut-gauche : chemins de fichiers (le Template mis en évidence, chemin "sensible" dont toute la génération dépend ; les Legends avec la mention "to check", voir ci-dessous). Haut-droite : nettoyage des dossiers de données (voir ci-dessous). Bas-gauche : liste des CSV. Bas-droite : progression + accès au dernier rapport. Les boutons "Generate report"/"Close" restent au niveau de la fenêtre, visibles quel que soit l'onglet actif.

Dossier des légendes : déplacé hors de DATA_ROOT. `LEGEND_FOLDER` (`00_constants.py`) pointe vers `<projet>/user_files/legend` plutôt que `data/legend`. Ce dossier n'est plus créé automatiquement par le script — il est entretenu manuellement par l'ingénieur dans les fichiers du projet Ansys, ce script ne fait que le lire. Un avertissement console signale son absence au chargement, comme pour le template. N'étant plus dans `DATA_ROOT`, il n'apparaît plus dans les tuiles de nettoyage.

Nettoyage des dossiers de données. Les tuiles sont générées dynamiquement à partir de `list_data_cleanup_folders()` (`00_constants.py`), qui liste tous les sous-dossiers de `DATA_ROOT` à l'exception de `LEGEND_FOLDER` — un nouveau sous-dossier de données ajouté au code obtient automatiquement sa tuile de nettoyage, sans changement côté UI. La taille et le nombre de fichiers de chaque tuile viennent de `get_folder_stats()`, un parcours récursif (`os.walk`) recalculé à l'ouverture de l'application et après chaque étape de génération (`_update_generation_progress`) — son coût croît avec le nombre de fichiers présents. `clear_folder_contents()` supprime le contenu d'un dossier sans supprimer le dossier lui-même, ce qui évite d'avoir à le recréer (`ensure_folder_exists`) à la génération suivante. Le bouton "Clear" par tuile et le bouton global "Delete all" appellent tous deux cette même fonction — une seule implémentation de suppression. Les deux actions sont bloquées derrière une confirmation (`MessageBoxButton.YesNo`), la suppression étant irréversible (pas de corbeille). Si le dossier des rapports est vidé, la tuile de statut du dernier rapport repasse à l'état neutre (`_reset_report_status_tile`), le fichier qu'elle référençait n'existant plus.

Listes CSV et rapport : "Show in folder" plutôt qu'un téléchargement. La grille de tuiles CSV est devenue une liste tabulaire (une ligne par fichier, nom à gauche + boutons à droite). Les boutons de téléchargement (`SaveFileDialog`) ont été retirés partout, au profit d'un bouton **"Show in folder"** (`_on_show_in_folder`, `Process.Start("explorer.exe", "/select,\"<chemin>\")`), qui ouvre l'explorateur Windows avec le fichier déjà sélectionné. Le bouton de visualisation s'appelle "Open" (`Process.Start(path)`).

`AnslysReportGenerator_WPF.xaml` ne contient que de la mise en page déclarative : consulter directement ce fichier pour l'apparence exacte, ou §7 ci-dessous pour son fonctionnement.

7. XAML : mise en page déclarative et lien avec Python

Ce qu'est le XAML ici. `AnslysReportGenerator_WPF.xaml` ne contient que de la mise en page déclarative : styles, couleurs (`Brush`), et les contrôles fixes de la fenêtre (onglets, cartes, boutons de la barre d'outils...), chacun identifié par un `x:Name`. Contrairement à un projet WPF "classique" (XAML compilé, associé à un `x:Class` et un fichier code-behind généré automatiquement), ce projet charge le XAML **à l'exécution** via `XamlReader.Load` : pas de compilation, pas de classe partielle, aucun attribut `Click="..."` dans le XAML. Toute la logique — construction dynamique des listes, câblage des événements — est écrite en Python.

Le lien XAML ↔ Python. Chaque contrôle nommé dans le XAML est retrouvé côté Python par `self.window.FindName("NomExact")`, centralisé dans `ReportGeneratorApp._find_controls` (ex : `btnGenerate` → `self.btn_generate`). Les gestionnaires d'événements sont ensuite câblés explicitement en Python (`ReportGeneratorApp._wire_events`), par exemple `self.btn_generate.Click += self._on_generate` — rien de tout cela n'apparaît dans le XAML.

Piège : NameScope des ControlTemplate. Un élément défini à l'intérieur d'un `ControlTemplate` (une `Style`) vit dans un `NameScope` séparé de celui de la fenêtre : `FindName()` ne le voit pas, même avec un `x:Name` correctement posé. C'est le cas du logo de la carte de crédit (bas de la colonne d'onglets, défini dans le `ControlTemplate` du style `TabControl`) : il est résolu via une `DynamicResource` (`SidebarLogoBitmap`) plutôt qu'un `x:Name`, la ressource étant peuplée depuis Python (`self.window.Resources["SidebarLogoBitmap"] = bitmap`, voir `_load_logo`) — un `ResourceDictionary` reste accessible comme un simple dictionnaire, quel que soit l'endroit de l'arbre visuel où il est défini.

Ajouter un nouveau bouton, étape par étape :

1. **Repérer l'emplacement** dans le XAML à partir d'un bouton similaire déjà existant : bouton de barre d'outils (`btnDeleteFigures`, `btnResetLegends...`), bouton d'en-tête de section (`btnZoneCheckXxx/btnZoneConfigXxx`), ou bouton de bas de fenêtre (`btnGenerate/btnClose`).
2. **Ajouter l'élément** dans le `StackPanel/DockPanel` concerné, en réutilisant un style existant plutôt que d'en définir un nouveau :

```
<Button x:Name="btnMyButton" Content="My action" Style="{StaticResource
SecondaryButton}"/>
```

Styles disponibles : `PrimaryButton`, `SecondaryButton`, `MiniButton`, `DangerButtonLight`, `DangerButtonStrong`.

3. **Valider le XAML** avant de tester dans Mechanical — une erreur de balisage échoue de façon peu explicite au chargement :

```
[xml]$doc = Get-Content -Raw "AnsysReportGenerator_WPF.xaml"
```

4. **Récupérer le contrôle côté Python**, dans `_find_controls` :

```
self.btn_my_button = self.window.FindName("btnMyButton")
```

5. **Écrire le handler**, puis le **câbler** dans `_wire_events` :

```
def _on_my_button_click(self, sender, e):
    ...

self.btn_my_button.Click += self._on_my_button_click
```

8. Notions métier Ansys utilisées dans le code

Terme	Signification	Où dans le code
Step / Load Case	Étape de chargement d'une analyse (ex : Step 1 = préchargement, Step 2 = charge de service)	<code>get_step_count</code> , <code>selected_steps</code> , <code>evaluate_result_for_step</code>
Boundary Condition (BC)	Contrainte/charge imposée (encastrement, pression, force...)	<code>collect_boundary_conditions[_multi]</code> , <code>build_bc_slides</code>
Bolt Pretension	Précontrainte de boulon	<code>collect_bolt_pretensions[_multi]</code> , <code>build_bp_slides</code>
Contact Tool	Analyse de la qualité d'un contact (gap, pression, glissement...) — existe en double : un dans <i>Connections</i> (définition, sans step) et un dans <i>Solution</i> (résultats, avec steps), distingués par leur position dans l'arbre (<code>_is_descendant_of</code>)	<code>collect_contact_tool_results</code> vs <code>collect_connection_contact_tool_results</code>
Bolt Tool	Efforts dans les connexions boulonnées (axial, cisaillement...)	<code>collect_bolt_tool_results[_multi]</code>

Terme	Signification	Où dans le code
Solution Information	Données de convergence du solveur ; ses enfants ("trackers") n'ont qu'un graphique 2D, pas de vue 3D	<code>collect_solution_information_trackers</code> , <code>export_chart_image_from_csv</code>
Named View	Vue caméra enregistrée dans le View Manager	<code>collect_views</code> , <code>apply_view_if_exists</code>
Section Plane	Plan de coupe pour révéler l'intérieur du modèle	<code>collect_section_planes</code> , <code>apply_section_plane</code>
Focus	Résultat agrégé filtré par sélection (non encore intégré dans l'UI active)	—

9. API Ansys Mechanical utilisées

Accès au modèle

```

ExtAPI.DataModel.Project.Model          # racine du modele
ExtAPI.DataModel.Project.Model.Analyses # liste des analyses du projet
ExtAPI.DataModel.AnalysisList           # idem, raccourci equivalent
ExtAPI.DataModel.GetObjectsByType(DataModelObjectCategory.XXX) # recherche par
categorie dans tout l'arbre (BC, Bolt Pretension, Contact Tool, Bolt Tool, Contact
Region, Figure...)
ExtAPI.DataModel.Project.Model.GetChildren(DataModelObjectCategory.Body, True) #
tous les corps (True = recursif)
ExtAPI.DataModel.Project.Model.Geometry # racine Geometrie, portee par
MeshPartRowConfig pour la case Geometry (voir §6)
ExtAPI.DataModel.Project.Model.Mesh     # racine Maillage, portee par
MeshPartRowConfig pour la case Mesh (voir §6)
ExtAPI.DataModel.Project.ProjectDirectory # dossier "<NomProjet>_files" du
projet courant - localise PROJECT_DIR (§2)

```

Affichage et capture d'image

```

ExtAPI.Graphics.Camera.SetFit()          # cadre la camera sur l'objet actif
- utilise uniquement par create_basic_views() (voir §6)
ExtAPI.Graphics.ExportImage(path, GraphicsImageExportFormat.PNG, settings) #
settings = Ansys.Mechanical.Graphics.GraphicsImageExportSettings()
ExtAPI.Graphics.ViewOptions.ModelColoring # coloration par materiau
ExtAPI.Graphics.ViewOptions.ShowMesh      # affichage du maillage
ExtAPI.Graphics.ViewOptions.ShowLogo      # logo Ansys - toujours desactive
(voir §6)
ExtAPI.Graphics.ViewOptions.ResultPreference.ContourView # mode
d'affichage des couleurs de resultat (§6) - lu/ecrit via getattr() (§11)
ExtAPI.Graphics.ViewOptions.ResultPreference.ScopingDisplay # mode
d'affichage du scoping (§6)

```

```

ExtAPI.Graphics.ViewOptions.ResultPreference.DeformationScaling      # mode
d'echelle de deformation (Auto/UserDefined)
ExtAPI.Graphics.ViewOptions.ResultPreference.DeformationScaleMultiplier # facteur
d'echelle (manuel ou x1/x2)
MechanicalEnums.Graphics.ScopingDisplay          # enum correspondant : ScopedBodies
/ ResultOnly / AllBodies
MechanicalEnums.Graphics.DeformationScaling      # enum correspondant : Auto /
UserDefined
ExtAPI.Graphics.GlobalLegendSettings.LegendOrientation #
LegendOrientationType.Vertical / .Horizontal
ExtAPI.Graphics.ImportLegend(path, unit)          # applique un fichier de legende
.xml - l'unité doit correspondre a celle de l'objet ACTUELLEMENT ACTIF (Activate())
systematique juste avant)
ExtAPI.Graphics.ModelViewManager.ExportModelViews(path) # liste les vues nommees,
en XML
ExtAPI.Graphics.ModelViewManager.ApplyModelView(view) # active une vue nommee
ExtAPI.Graphics.ModelViewManager.Capture3DImage(path) # export .avz (vue 3D
interactive) de l'objet actif - bouton "Export to 3D"
ExtAPI.Graphics.SectionPlanes                      # plans de coupe disponibles
(apply_section_plane)
ExtAPI.Graphics.Redraw()                            # force le rafraichissement du
viewport - obligatoire apres tout changement de propriete d'affichage par script

```

Résultats et steps

```

SetDriverStyle.ResultSet          # + .SetNumber : repositionne un
resultat sur un step donne avant reevaluation (evaluate_result_for_step)
obj.Activate()                    # active l'objet dans le viewport
- prerequisite a la plupart des captures/exports
obj.Name / obj.Children / obj.Parent / obj.DataModelObjectCategory # disponibles
sur la plupart des objets individuels
obj.AddFigure()                   # puis figure.Activate() :
snapshot Figure, capture fiable preferee a une capture "live" directe

```

Autres API notables

```

Ansys.ACT.Mechanical.Transaction # "with Transaction(True): ..." - differe le
rafraichissement UI pendant des operations en masse (suppression de figures,
boucle sur tous les corps...)
materials.GetMaterialPropertyByName(material, group) # module Ansys pour lire
les proprietes materiau

```

Côté .NET / COM (hors API Ansys)

```

clr.AddReference("Microsoft.Office.Interop.PowerPoint") # + "Office"
clr.AddReference("System.Windows.Forms") / "System.Drawing"

```

```
clr.AddReference("PresentationFramework") / "PresentationCore" / "WindowsBase" #
WPF
```

10. Comment le code pilote PowerPoint (COM Interop)

Le projet ne dépend d'aucune bibliothèque Python pour manipuler PowerPoint : `python-pptx` (comme `pandas` ou `openpyxl`) est incompatible avec IronPython 2.7, le moteur Python embarqué dans Ansys Mechanical, et n'est donc jamais utilisé ici. Il pilote directement l'application PowerPoint installée sur le poste via **COM Interop** : Microsoft Office expose une API COM, et .NET fournit des assemblées "Interop" (`Microsoft.Office.Interop.PowerPoint`, `Office`) qui traduisent cette API COM en classes .NET utilisables depuis n'importe quel langage .NET — donc depuis IronPython, qui tourne lui-même sur le CLR .NET. C'est ce que fait `03_ppt_utils.py` en tout début de fichier :

```
clr.AddReference("Microsoft.Office.Interop.PowerPoint")
clr.AddReference("Office")
import Microsoft.Office.Interop.PowerPoint as PPT
import Microsoft.Office.Core as Office
```

`clr.AddReference` charge l'assembly .NET correspondante (installée avec Office, indépendamment du projet), après quoi `PPT` et `Office` s'utilisent comme des modules Python normaux — à ceci près que chaque objet manipulé (`Presentation`, `Slide`, `Shape`...) est en réalité un objet COM distant : chaque accès à une propriété ou chaque appel de méthode interroge réellement le processus PowerPoint en cours d'exécution, ce qui a un coût (d'où plusieurs optimisations décrites plus bas).

Toute la logique est concentrée dans la classe `PPTReportBuilder`, qui possède une session PowerPoint unique pour toute la génération du rapport (une seule ouverture/fermeture, pas une par slide). Son constructeur illustre le principe central du module :

```
def __init__(self, template_path):
    self.working_copy_path = get_unique_file_path(
        REPORT_OUTPUT_FOLDER, _build_working_copy_base_name(), ".pptx")
    shutil.copyfile(template_path, self.working_copy_path)

    self.app = PPT.ApplicationClass()
    self.app.Visible = True
    self.presentation = self.app.Presentations.Open(self.working_copy_path,
        WithWindow=True)
```

`PPT.ApplicationClass()` démarre (ou récupère) une instance de l'application PowerPoint elle-même, exactement comme si l'utilisateur avait double-cliqué sur son icône ; `self.app.Presentations.Open(...)` y ouvre ensuite un fichier, ce qui renvoie un objet `Presentation` sur lequel toutes les opérations suivantes portent. Le template original n'est jamais ouvert directement : une copie (`working_copy_path`) est créée juste avant via `shutil.copyfile`, et c'est cette copie qui est ouverte — un `Ctrl+S` accidentel dans la fenêtre PowerPoint pendant la génération écrase donc la copie, jamais le template corporate. `self.app.Visible = True` n'est pas cosmétique : une session laissée invisible s'est révélée instable sur un rapport avec beaucoup

de slides (l'objet `SlideMaster` finissait par devenir inaccessible en cours de génération), donc la fenêtre PowerPoint reste visible pendant toute la génération et se referme normalement à la fin, dans `close()`.

Ajouter une slide consiste toujours à demander un layout personnalisé du template par son index, puis à insérer cette slide à la fin de la présentation :

```
def _add_slide(self, layout_index):
    layout = self.presentation.SlideMaster.CustomLayouts[layout_index]
    return self.presentation.Slides.AddSlide(self.presentation.Slides.Count + 1,
    layout)
```

`SlideMaster.CustomLayouts` est la liste des layouts personnalisés définis dans le template (visible dans PowerPoint via Affichage > Masque des diapositives) ; leur index (`LAYOUT_IMAGE_TABLE = 10`, etc., dans `00_constants.py`) est déterminé une fois pour toutes en listant les layouts du template (voir §13) et ne change plus tant que le template n'est pas modifié. `add_image_table_slide` illustre ensuite comment une zone de la slide est remplie une fois celle-ci créée :

```
slide.Shapes[8].TextFrame.TextRange.Text = comment
slide.Shapes[2].TextFrame.TextRange.Text = title
...
coord = self.presentation.SlideMaster.CustomLayouts[LAYOUT_IMAGE_TABLE].Shapes[3]
slide.Shapes.AddPicture(img_path, Office.MsoTriState.msoFalse,
Office.MsoTriState.msoTrue,
                        coord.Left, coord.Top, coord.Width, coord.Height)
```

Chaque `Shapes[n]` correspond à une zone précise définie dans le layout au moment de sa création dans PowerPoint (une zone de titre, une zone d'image, une zone de table...) ; l'ordre et l'index de ces zones sont figés par le template, pas par le code, d'où l'importance de ne jamais réorganiser les zones d'un layout existant sans mettre à jour les index utilisés dans `03_ppt_utils.py` (voir §13). Le texte est toujours affecté sur la slide nouvellement créée, jamais sur le layout lui-même : modifier le layout modifierait le master template pour toutes les slides futures. Pour positionner l'image, le code va chercher les coordonnées (`Left`, `Top`, `Width`, `Height`) de la zone d'image telle que définie *dans le layout*, plutôt que de coder ces coordonnées en dur : la position et la taille de l'image restent ainsi cohérentes avec ce qui a été dessiné dans le template, même si celui-ci évolue.

`add_csv_table` est la partie la plus sensible aux performances, car chaque instruction du bloc suivant est un aller-retour COM :

```
for r in range(1, rows + 1):
    row_cells = table.Rows(r).Cells
    for border_index in range(1, 5):
        row_cells.Borders(border_index).ForeColor.RGB = 0x000000
        row_cells.Borders(border_index).Weight = 1
```


Les bordures sont posées une fois par ligne entière (`table.Rows(r).Cells` accepte une plage de cellules) plutôt que cellule par cellule × côté par côté, ce qui a divisé par le nombre de colonnes le temps de formatage d'un tableau (jusqu'à 45 secondes pour 8 lignes avant cette optimisation, contre une fraction de seconde après). Le texte et la police, eux, n'ont pas d'équivalent "par plage" dans l'API COM de PowerPoint et restent donc nécessairement posés cellule par cellule dans la boucle suivante. Une dernière particularité : après avoir rempli le tableau, le code force `table.Rows(r).Height = 1` sur chaque ligne — une valeur volontairement absurde, mais PowerPoint la ramène automatiquement à la hauteur minimale réellement nécessaire pour loger le texte, ce qui est le seul moyen de resserrer un tableau déjà créé (`AddTable` alloue par défaut une hauteur bien supérieure au nécessaire pour du texte en taille 7, ce qui ferait déborder la slide sans ce correctif).

Enfin, `close()` illustre la règle à respecter systématiquement avec les objets COM : les libérer explicitement plutôt que de compter sur le ramasse-miettes Python, pour ne jamais laisser un processus PowerPoint invisible tourner en arrière-plan après une erreur :

```
def close(self):
    self.presentation.Save()
    self.presentation.Close()
    self.app.Quit()
```

11. Raisonnements Python employés dans le projet

Plusieurs choix récurrents dans le code répondent à des contraintes propres à IronPython 2.7 et à l'exécution dans la console de scripting Mechanical ; les comprendre aide à lire (et à étendre) n'importe quel module du projet.

Chargement par `execfile()` plutôt que par `import`. `AnsysReportGenerator_WPF.py` ne fait pas `import constants` ou `from data_export import ...` : il appelle `execfile(module_path)` pour chacun des six modules, dans l'ordre. `execfile()` exécute le contenu d'un fichier comme s'il avait été tapé directement à la suite dans la même console, dans le même espace de noms global — contrairement à `import`, qui créerait un espace de noms séparé (`data_export.export_active_tabular_data` au lieu de `export_active_tabular_data`). C'est ce partage volontaire d'un seul espace de noms global qui permet à `05_interactive_slides.py` d'appeler directement `export_active_tabular_data` (définie dans `01_data_export.py`) sans préfixe, exactement comme le fait la console de scripting Mechanical elle-même vis-à-vis de `ExtAPI/DataModel`. C'est aussi ce qui permet à une fonction définie plus tôt de référencer un nom défini plus tard dans un autre module : `00_constants.py` utilise `PROJECT_DIR`, qui n'est en réalité défini que dans `AnsysReportGenerator_WPF.py`, avant l'`execfile()` de `00_constants.py` — l'ordre de chargement (`00` → `05`, puis le script principal en dernier dans la console) est donc significatif et ne doit jamais être changé.

Accès aux enums .NET par `getattr()` plutôt que par `import explicite`. Plusieurs endroits du code, par exemple `apply_contour_view` dans `05_interactive_slides.py`, écrivent :

```
vo.ResultPreference.ContourView = getattr(vo.ResultPreference.ContourView,
    contour_view)
```


au lieu d'importer l'énumération `.NET` correspondante et d'écrire une longue chaîne `if/elif` pour convertir la chaîne choisie dans l'UI ("`ContourBands`", "`Isolines`"...) en valeur d'enum. `getattr(objet, "NomDeMembre")` va chercher l'attribut nommé "`NomDeMembre`" sur le *type* de `objet` (ici le type de l'enum `ContourView` déjà présent sur l'instance courante) : comme les chaînes de caractères utilisées dans les listes déroulantes de l'UI (`CONTOUR_VIEW_OPTIONS`) portent exactement les mêmes noms que les membres de l'enum `.NET`, `getattr` fait directement la conversion chaîne → valeur d'enum en une ligne, sans avoir à importer explicitement chaque type d'enum ni à le tenir à jour si Ansys en ajoute un membre dans une version future.

Échec local, jamais d'exception qui remonte jusqu'à l'UI. Presque toutes les fonctions d'export ou d'application de réglage suivent le même schéma :

```
try:
    ...
except Exception as e:
    print "Error: " + str(e)
    return False # ou None
```

Ce choix est délibéré : une génération de rapport peut porter sur des dizaines de slides, et une seule ligne de Boundary Condition mal configurée (ou une image qui échoue à s'exporter) ne doit pas interrompre toute la génération et faire perdre le travail déjà fait sur les slides précédentes. L'erreur est donc absorbée localement, journalisée dans la console de scripting (visible par l'ingénieur), et la fonction renvoie une valeur "neutre" (`False`, `None`, ou simplement ne fait rien) que l'appelant peut tester pour décider de continuer ou non.

Les fonctions "`collect_`" renvoient toujours une liste Python simple. Que la source soit `ExtAPI.DataModel.GetObjectsByType(...)`, un parcours d'arbre récursif, ou la compilation de plusieurs analyses (variantes `_multi`), chaque collecteur renvoie une `list` Python ordinaire, jamais l'objet `.NET/COM` d'origine. Cela découple complètement l'interface WPF (qui construit ses listes déroulantes et cases à cocher à partir de ces listes) du détail de la façon dont chaque catégorie d'objet est retrouvée dans l'arbre Mechanical — un nouveau collecteur peut changer entièrement sa logique interne sans que le code de l'UI qui le consomme ait à changer.

Constantes et options (`libellé`, `valeur`). Les options destinées à apparaître dans l'UI (`CONTOUR_VIEW_OPTIONS`, `LEGEND_ORIENTATION_OPTIONS`, `DEFORMATION_SCALE_MODE_OPTIONS`, `BASIC_VIEW_ORIENTATIONS`...) sont systématiquement des listes de tuples (`libellé affiché en anglais dans l'UI`, `valeur technique utilisée dans le code/l'API`), avec des fonctions `xxx_label/xxx_from_label` symétriques pour convertir dans un sens ou dans l'autre. Cela sépare proprement ce qui est montré à l'ingénieur (modifiable sans risque) de ce qui doit rester identique au nom exact attendu par l'API `.NET`.

12. Bases Python illustrées par le code du projet

Cette section reprend les briques de base du langage Python (compatibles IronPython 2.7) à partir d'exemples réels du projet, pour un lecteur qui découvrirait Python via ce code.

Variables et types. Une variable n'a pas de type déclaré, elle prend le type de ce qu'on lui affecte : `DATA_ROOT = os.path.join(PROJECT_DIR, "data")` crée une variable `DATA_ROOT` de type `str`. `MAX_TABLE_ROWS =`

`50` crée un entier. Une liste se note entre crochets et peut grandir dynamiquement : `_MODULE_FILES = ["00_constants.py", "01_data_export.py", ...]`. Un dictionnaire associe des clés à des valeurs entre accolades ; `_DEFAULT_FILE_PATHS = dict((name, globals()[name]) for name, _, _, _ in FILE_PATH_SETTINGS)` en construit un à la volée à partir d'une liste de tuples.

Fonctions. `def` définit une fonction, ses paramètres entre parenthèses, et `return` renvoie sa valeur de sortie (une fonction sans `return` renvoie implicitement `None`) :

```
def safe_file_name(name):
    return re.sub(r'[\\/:*?"<>|]', "_", name).strip() or "object"
```

Un paramètre peut avoir une valeur par défaut, utilisée si l'appelant ne le fournit pas : `def add_image_table_slide(self, title, subtitle, img_path=None, csv_path=None, comment=" ")` peut donc être appelée avec seulement `title` et `subtitle`, `img_path` valant alors `None`. Le projet utilise systématiquement des chaînes formatées avec `.format()`, jamais les f-strings de Python 3.6+ (indisponibles en IronPython 2.7) : `"result_{}.csv".format(step_id)` plutôt que `f"result_{step_id}.csv"`.

Classes. `class NomDeClasse(object)` : définit une classe (le `(object)` explicite est nécessaire en Python 2 pour obtenir une classe "nouveau style"). `__init__` est le constructeur, appelé automatiquement à la création d'une instance ; `self` (premier paramètre de toute méthode) désigne l'instance elle-même et doit être utilisé explicitement pour lire ou écrire un attribut :

```
class PPTReportBuilder(object):
    def __init__(self, template_path):
        self.working_copy_path = get_unique_file_path(...)
        self.app = PPT.ApplicationClass()

    def save_as(self, output_path):
        self.presentation.SaveAs(output_path)
```

`self.app` et `self.working_copy_path` sont des attributs propres à chaque instance de `PPTReportBuilder` : deux instances créées séparément auraient chacune leur propre session PowerPoint, sans interférence. Une méthode s'appelle ensuite sur une instance : `builder = PPTReportBuilder(TEMPLATE_PATH)` puis `builder.save_as(output_path)`.

Boucles et conditions. `for` itère sur n'importe quelle séquence (liste, plage de nombres, résultat d'une requête API) ; `range(1, rows + 1)` produit les entiers de 1 à `rows` inclus (Python exclut toujours la borne haute de `range`). `if/elif/else` teste des conditions ; l'indentation (toujours 4 espaces dans ce projet, jamais de tabulations mélangées) délimite les blocs, il n'y a pas d'accolades en Python :

```
for step_id in steps:
    if step_id in selected_steps:
        rows.append(evaluate_result_for_step(obj, step_id))
    else:
        print "Step skipped: {}".format(step_id)
```

Gestion d'erreurs (`try/except`). Un bloc `try` exécute du code potentiellement risqué (accès COM, accès disque, appel API Ansys) ; si une exception est levée, l'exécution saute directement au bloc `except` correspondant plutôt que de faire planter tout le script :

```
try:
    graphics.ExportImage(output_path, export_settings)
    return True
except Exception as e:
    print "Error exporting view: {}".format(e)
    return False
```

`except Exception as e` capture toute erreur standard et la rend disponible dans la variable `e` (généralement convertie en texte via `str(e)` pour l'afficher). Ce schéma est omniprésent dans le projet (voir §11).

Gestionnaires de contexte (`with`). `with open(path, "rb") as f:` ouvre un fichier et garantit sa fermeture automatique à la sortie du bloc, même en cas d'erreur à l'intérieur — équivalent plus sûr et plus court qu'un `try/finally` manuel avec `f.close()`. Utilisé pour tous les accès fichier CSV du projet, et détourné pour un usage différent avec `with Transaction(True): ... (Ansys.ACT.Mechanical.Transaction)`, qui ne gère pas un fichier mais diffère le rafraîchissement de l'interface Mechanical jusqu'à la sortie du bloc, pour accélérer les opérations en masse.

Compréhensions de liste. Une compréhension construit une nouvelle liste en une seule expression, plus concise qu'une boucle `for` classique avec `append` : `[cell.decode("utf-8") for cell in row]` (dans `add_csv_table`) relit chaque cellule d'une ligne CSV et la décode de UTF-8, en produisant directement la liste décodée.

`import` vs `execfile`. Le projet utilise `import` pour les modules standards (`import csv, import os`) et les assemblées .NET (`import Microsoft.Office.Interop.PowerPoint as PPT`, après `clr.AddReference`), mais `execfile()` pour charger ses propres modules 00 à 05 — voir §11 pour l'explication de ce choix inhabituel, spécifique au contexte d'exécution dans la console Mechanical.

13. Créer une nouvelle slide personnalisée dans le Master Template

Ajouter un nouveau type de slide au rapport suppose d'abord de créer le layout correspondant dans le template PowerPoint lui-même, puis seulement ensuite d'écrire le code Python qui le remplit. La procédure côté PowerPoint est stricte sur un point : **le nouveau layout doit toujours être inséré à la fin du masque de diapositives, jamais au milieu**. Tous les index utilisés dans le code (`LAYOUT_IMAGE_TABLE = 10`, `LAYOUT_TABLE_ONLY = 8`, `LAYOUT_MESH_MULTII = 11`, dans `00_constants.py`) correspondent à la position du layout dans la liste `CustomLayouts` du template ; insérer un nouveau layout au milieu de cette liste décale l'index de tous les layouts existants situés après lui, et casse silencieusement toutes les slides déjà générées par le code actuel.

Pour créer le layout, ouvrez le Master Template dans PowerPoint (Affichage > Masque des diapositives), insérez une nouvelle mise en page à la suite des layouts existants, et construisez son contenu soit en dessinant de nouvelles zones (zone de texte, zone d'image, tableau), soit en copiant des éléments d'un layout existant proche de ce qui est recherché. Une fois le layout terminé, enregistrez cette nouvelle version du template sous un **nom différent** de l'original (par exemple en ajoutant un suffixe), afin de conserver une

copie de secours du template actuellement utilisé en production si la modification s'avère incompatible avec le code existant.

Il faut ensuite identifier l'index du nouveau layout ainsi que l'index de chacune de ses zones (*Shapes*), car c'est par ces index que le code Python les désigne (voir *Shapes[n]* au §10). Cela se fait en exécutant, dans la console de scripting de Mechanical, un petit script qui ouvre le template via COM Interop exactement comme le fait *PPTReportBuilder*, et qui liste les layouts disponibles :

```
import clr
import os
import System

clr.AddReference("Microsoft.Office.Interop.PowerPoint")
clr.AddReference("Office")
import Microsoft.Office.Interop.PowerPoint as PPT
import Microsoft.Office.Core as Office
from Microsoft.Office.Core import MsoTriState

app = PPT.ApplicationClass()
app.Visible = True
template_path = r"CHEMIN_VERS_LE_TEMPLATE.pptx" # a adapter
presentation = app.Presentations.Open(template_path, WithWindow=True)
custom_layouts = presentation.SlideMaster.CustomLayouts

for design in presentation.Designs:
    for i in range(1, design.SlideMaster.CustomLayouts.Count + 1):
        layout = design.SlideMaster.CustomLayouts[i]
        print(i, layout.Name)
```

Ce premier bloc affiche la liste complète des layouts existants avec leur index et leur nom (par exemple (1, "Page de Titre"), (10, "Image + Table")...) : c'est là qu'on repère l'index attribué au nouveau layout qui vient d'être ajouté à la fin. Une fois cet index repéré, on sélectionne ce layout puis on liste ses zones dans l'ordre où PowerPoint les connaît :

```
slide = custom_layouts[10] # remplacer par l'index du nouveau layout

index = 0
for shape in slide.Shapes:
    index += 1
    print shape.Name
```

Ce second bloc donne, pour chaque zone du layout, son nom et sa position dans la collection *Shapes* (le premier élément listé correspond à *Shapes[1]*) : c'est cette correspondance entre position et rôle visuel de la zone (titre, image, table, commentaire...) qui doit ensuite être reportée dans le code Python, exactement comme *LAYOUT_IMAGE_TABLE* est aujourd'hui documenté en commentaire dans *00_constants.py* (# *title[2] / subtitle[4] / image[3] / table[1] / comment[8]*). Une nouvelle fonction

`add..._slide` peut alors être ajoutée à `03_ppt_utils.py` sur le modèle de `add_image_table_slide`, en utilisant l'index du nouveau layout et les index de zones ainsi identifiés.

14. Pièges connus / choix techniques

- **Contraintes IronPython 2.7** : le moteur de script embarqué dans Ansys Mechanical exécute du Python 2.7 via .NET, pas du Python 3. Toute modification du code doit donc rester compatible avec ces restrictions :
 - `.format()` à la place des f-strings : `"result_{}.csv".format(step_id)`, jamais `f"result_{step_id}.csv"` (erreur de syntaxe en IronPython 2.7).
 - `print "texte"` en tant qu'instruction, jamais `print("texte")` en tant que fonction.
 - `os.path.join(...)` à la place du module `pathlib`, absent d'IronPython 2.7.
 - Pas d'annotations de type (`variable: str = ""`), pas de `async/await`.
 - Les bibliothèques `pandas`, `openpyxl` et `python-pptx` sont incompatibles et ne doivent jamais être importées — c'est la raison pour laquelle toutes les données tabulaires du projet transitent par de simples fichiers CSV (module standard `csv`) plutôt que par ces bibliothèques.
- **Session PowerPoint toujours visible** (`self.app.Visible = True` dans `PPTReportBuilder.__init__`) : une session laissée invisible s'est révélée instable sur un rapport avec beaucoup de slides (`COMException` sur `SlideMaster` en cours de génération). La fenêtre PowerPoint se referme normalement à la fin (`close()`).
- **Le template original n'est jamais ouvert directement** — toujours une copie de travail (voir §4), pour ne jamais risquer de l'écraser via un `Ctrl+S` accidentel pendant la génération.
- **Bordures de table posées par ligne entière**, pas cellule par cellule × côté par côté : chaque aller-retour COM est coûteux, cette optimisation a divisé par ~N (N = nb colonnes) le temps de formatage d'un tableau.
- **Unité de légende toujours déduite dynamiquement** (`get_result_display_unit`, lit le texte affiché dans `VisibleProperties`, pas `result_obj.Maximum.Unit` jugé peu fiable) : `ImportLegend()` compare l'unité demandée à celle de l'objet **actuellement actif** dans le viewport, d'où un `Activate()` explicite systématique juste avant, pour éviter un décalage d'une ligne avec l'objet réellement traité.
- **CSV toujours lu/écrit en UTF-8 explicite** (`open(path, "rb")` + décodage manuel) : les unités renvoyées par Mechanical contiennent parfois des caractères spéciaux (degré, micro...) qui font planter une lecture/écriture sans encodage explicite.
- **Limite d'affichage des tableaux** (`MAX_TABLE_ROWS` / `MAX_TABLE_COLUMNS`, 50×50 par défaut) : au-delà, le CSV est quand même généré mais n'est pas inséré comme table PowerPoint (illisible une fois inséré).
- **Insertion de layout dans le template** : toujours à la fin du masque de diapositives, jamais au milieu — voir §13 pour la procédure complète et la raison (décalage des index `LAYOUT_*` utilisés partout dans le code).